

Sri Lanka Institute of Information Technology

IT2140-Database Design and Development



Year 2 Semester 1

Web-based pet care system_DDD_Jan_I_31

Nimalika G.V.D-IT23385832

Part A – Mapping EER to Relational Schema

1. relational schema: - **Relational Schema:**

Customer	<u>customerID</u>	fullName	email	phone	address	password	status
----------	-------------------	----------	-------	-------	---------	----------	--------

Pet	<u>petID</u>	petName	petType	breed	age	gender	size	healthNotes	customerID (FK)
-----	--------------	---------	---------	-------	-----	--------	------	-------------	--------------------

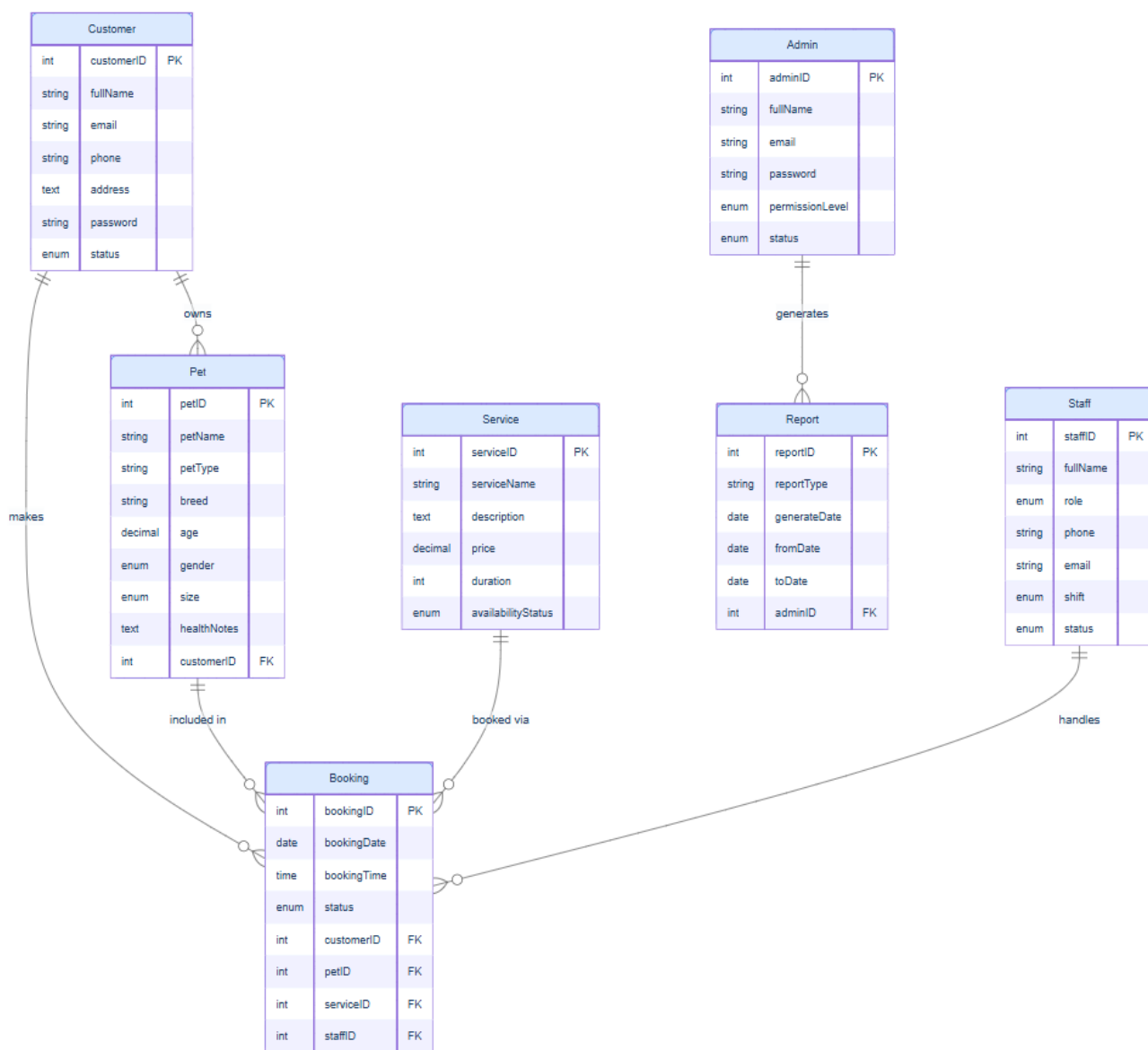
Service	<u>serviceID</u>	serviceName	description	price	duration	availabilityStatus
---------	------------------	-------------	-------------	-------	----------	--------------------

Staff	<u>staffID</u>	fullName	role	phone	email	shift	status
-------	----------------	----------	------	-------	-------	-------	--------

Booking	<u>bookingID</u>	bookingDate	bookingTime	status	<i>customerID (FK)</i>	<i>petID (FK)</i>	<i>serviceID (FK)</i>	<i>staffID (FK)</i>
---------	------------------	-------------	-------------	--------	----------------------------	-----------------------	---------------------------	-------------------------

Admin	<u>adminID</u>	fullName	email	password	permissionLevel	status
-------	----------------	----------	-------	----------	-----------------	--------

Report	<u>reportID</u>	reportType	generateDate	fromDate	toDate	<i>adminID (FK)</i>
--------	-----------------	------------	--------------	----------	--------	-------------------------



2. Relational Schema Diagram (MySQL Workbench Style):

The following tables represent the schema as it appears in MySQL Workbench, showing field names, data types, and PK/FK indicators:

PK	customer		PK	booking		PK	report	
PK	<u>CustomerID</u>	int (11)	PK	<u>BookingID</u>	int (11)	PK	<u>ReportID</u>	int (11)
			FK	CustomerID	int (11)	FK	AdminID	int (11)
			FK	PetID	int (11)		ReportType	varchar (100)
			FK	ServiceID	int (11)		GenerateDate	date
			FK	StaffID	int (11)			

	BookingDate	date		FromDate	date
	BookingTime	time		ToDate	date
	Status	enum('Pending','Confirmed','Completed','Cancelled')			

P K	pet		P K	staff		P K	admin	
P K	<u>PetID</u>	int(11)	P K	<u>StaffID</u>	int(11)	P K	<u>AdminID</u>	int(11)
F K	CustomerID	int(11)		FullName	varchar(100)		FullName	varchar(100)
	PetName	varchar(100)		Role	enum('Operations Manager', 'Staff Supervisor', 'Customer Support Officer', 'System Administrator')		Email	varchar(100)
	PetType	varchar(50)		Phone	varchar(15)		Password	varchar(255)
	Breed	varchar(100)		Email	varchar(100)		PermissionLevel	enum('Full','Limited')
	Age	decimal(4,1)		Shift	enum('Morning','Afternoon','Night')		Status	enum('Active','Inactive')
	Gender	enum('Male','Female','Unknown')		Status	enum('Active','Inactive')			
	Size	enum('Small','Medium','Large')						
	HealthNotes	text						

PK	service	
PK	<u>ServiceID</u>	int(11)
	ServiceName	varchar(100)
	Description	text
	Price	decimal(10,2)
	Duration	int(11) -- minutes
	AvailabilityStatus	enum('Available','Unavailable')

1. PKs, FKs, and constraints:-

Table	PK	FK(s)	Constraints
Customer	CustomerID	â€”	Email UNIQUE NOT NULL Status ENUM('Active','Inactive') DEFAULT 'Active'
Pet	PetID	CustomerID â†’ Customer(CustomerID)	ON DELETE CASCADE ON UPDATE CASCADE CustomerID NOT NULL
Service	ServiceID	â€”	Price >= 0 CHECK AvailabilityStatus ENUM ('Available','Unavailable')
Staff	StaffID	â€”	Email UNIQUE NOT NULL Role ENUM('Operations Manager', 'Staff Supervisor','Customer Support Officer','System Admin') Status ENUM('Active','Inactive')
Booking	BookingID	CustomerID â†’ Customer(CustomerID) PetID â†’ Pet(PetID) ServiceID â†’ Service(ServiceID) StaffID â†’ Staff(StaffID)	Status ENUM('Pending', 'Confirmed','Completed', 'Cancelled') DEFAULT 'Pending' UNIQUE(StaffID, BookingDate, BookingTime) â€” prevents double-booking (FR10) CreatedAt DEFAULT CURRENT_TIMESTAMP
Admin	AdminID	â€”	Email UNIQUE NOT NULL PermissionLevel ENUM ('Full','Limited') DEFAULT 'Full'
Report	ReportID	AdminID â†’ Admin(AdminID)	GenerateDate DEFAULT CURRENT_DATE CHECK(ToDate >= FromDate) AdminID NOT NULL

2. Mapping choices for ISA: -

- **User → Customer / Staff:** Implemented using primary key inheritance. CustomerID and StaffID are also UserID. The ISA relationship between the abstract User concept and the specialised Customer/Staff roles uses a single table per subtype approach:
 - No redundancy – shared user attributes (name, contact, role) are not duplicated.
 - Enforces 1:1 mapping between a user record and their role.
 - Allows easy querying of all users regardless of role.
 - Supports polymorphism in queries (role-based access control via the Role field).
- **Staff → Subtypes (Operations Manager, Staff Supervisor, Customer Support Officer, System Administrator):** These are NOT modelled as separate tables because:
 - They do not have unique attributes beyond those in Staff.
 - They are roles within Staff, captured via the Role column using ENUM.

- `ENUM('Operations Manager', 'Staff Supervisor', 'Customer Support Officer', 'System Administrator')` is sufficient to differentiate subtypes.

3. Schema Refinement:

- **Booking absorbs the Request relationship** – the M:1 Customer–Service relationship from the EER is fully absorbed into the Booking table via CustomerID and ServiceID FKs, eliminating the need for a separate junction table.
- **Double-booking prevention (FR10)** – a UNIQUE constraint on (StaffID, BookingDate, BookingTime) prevents the same staff member from being booked twice at the same date/time slot.
- **Added CHECK constraints** – Price ≥ 0 on Service, ToDate $\geq$ FromDate on Report.
- **Password hashing** – Password fields use VARCHAR(255) to store SHA2-256 hashed values, supporting NFR3 (Security).
- **CreatedAt timestamps** – added to Customer and Booking tables using DEFAULT CURRENT_TIMESTAMP for audit trail purposes.

Part B – SQL DDL Implementation

1. CREATE TABLE Statements: -

Customer Table

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0078 seconds.)

```
CREATE TABLE Customer (
customerID INT PRIMARY KEY, fullName VARCHAR(100) NOT NULL,
email VARCHAR(100) UNIQUE,
phone VARCHAR(15), address VARCHAR(255),
password VARCHAR(100) NOT NULL,
status VARCHAR(20) CHECK (status IN ('Active', 'Inactive')) );
```

Pet Table

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0010 seconds.)

```
CREATE TABLE Pet (
petID INT PRIMARY KEY,
name VARCHAR(100) NOT NULL,
type VARCHAR(50),
breed VARCHAR(50),
age INT CHECK (age >= 0),
gender VARCHAR(10),
healthStatus VARCHAR(100),
customerID INT, FOREIGN KEY (customerID) REFERENCES Customer(customerID) );
```

Service Table

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0010 seconds.)

```
CREATE TABLE Service (  
  serviceID INT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  description VARCHAR(255),  
  price DECIMAL(10,2) CHECK (price >= 0), duration INT,  
  availabilityStatus VARCHAR(20) );
```

Staff Table

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0012 seconds.)

```
CREATE TABLE Staff (  
  staffID INT PRIMARY KEY,  
  fullName VARCHAR(100),  
  role VARCHAR(50),  
  phone VARCHAR(15),  
  email VARCHAR(100),  
  shift VARCHAR(50) );
```

Operations Manager Table

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0005 seconds.)

```
CREATE TABLE OperationsManager (  
  staffID INT PRIMARY KEY,  
  FOREIGN KEY (staffID) REFERENCES Staff(staffID) );
```

Staff Supervisor Table

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0009 seconds.)

```
CREATE TABLE StaffSupervisor (  
  staffID INT PRIMARY KEY,
```

```
FOREIGN KEY (staffID) REFERENCES Staff(staffID) );
```

Customer support officer Table

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0008 seconds.)

```
CREATE TABLE CustomerSupportOfficer (  
  staffID INT PRIMARY KEY,  
  FOREIGN KEY (staffID) REFERENCES Staff(staffID) );
```

System Admin Table

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0005 seconds.)

```
CREATE TABLE SystemAdministrator (  
  staffID INT PRIMARY KEY,  
  FOREIGN KEY (staffID)  
  REFERENCES Staff(staffID) );
```

Booking Table

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0011 seconds.)

```
CREATE TABLE Booking (  
  bookingID INT PRIMARY KEY,  
  bookingDate DATE NOT NULL,  
  bookingTime TIME NOT NULL,  
  status VARCHAR(20),  
  customerID INT,  
  petID INT,  
  serviceID INT,  
  staffID INT,  
  FOREIGN KEY (customerID) REFERENCES Customer(customerID),  
  FOREIGN KEY (petID) REFERENCES Pet(petID),  
  FOREIGN KEY (serviceID) REFERENCES Service(serviceID),  
  FOREIGN KEY (staffID) REFERENCES
```

Admin Table

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0007 seconds.)

```
CREATE TABLE Admin (  
  adminID INT PRIMARY KEY,  
  fullName VARCHAR(100),  
  email VARCHAR(100),  
  password VARCHAR(100) permissionLevel VARCHAR(50), status VARCHAR(20) );
```

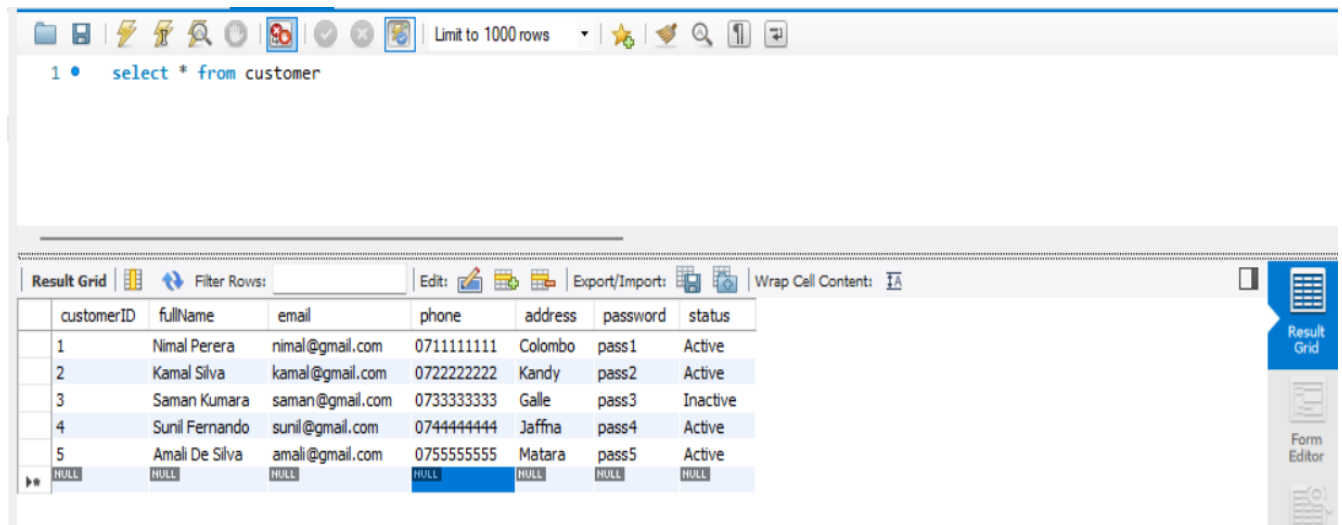
Report Table

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0018 seconds.)

```
CREATE TABLE Report (  
  reportID INT PRIMARY KEY,  
  reportType VARCHAR(50),  
  generatedDate DATE,  
  format VARCHAR(20),  
  adminID INT, staffID INT, FOREIGN KEY (adminID) REFERENCES Admin(adminID),  
  FOREIGN KEY (staffID) REFERENCES Staff(staffID) );
```

Part C – Insert Sample Data

1. Customer



1 • select * from customer

Limit to 1000 rows

Result Grid

	customerID	fullName	email	phone	address	password	status
1	1	Nimal Perera	nimal@gmail.com	0711111111	Colombo	pass1	Active
2	2	Kamal Silva	kamal@gmail.com	0722222222	Kandy	pass2	Active
3	3	Saman Kumara	saman@gmail.com	0733333333	Galle	pass3	Inactive
4	4	Sunil Fernando	sunil@gmail.com	0744444444	Jaffna	pass4	Active
5	5	Amali De Silva	amali@gmail.com	0755555555	Matara	pass5	Active
6	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Filter Rows: Edit: Export/Import: Wrap Cell Content: Form Editor

2. Pet

Limit to 1000 rows

1 • `SELECT * FROM pet;`

	petID	name	type	breed	age	gender	healthStatus	customerID
▶	1	Max	Dog	Labrador	3	Male	Healthy	1
	2	Kitty	Cat	Persian	2	Female	Healthy	2
	3	Rocky	Dog	Bulldog	4	Male	Injured	3
	4	Milo	Cat	Siamese	1	Male	Healthy	4
	5	Bella	Dog	Beagle	5	Female	Sick	5
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: `Ctrl+Z`

Result Grid
Form Editor

3. Service

Limit to 1000 rows

1 • `SELECT * FROM service;`

	serviceID	name	description	price	duration	availabilityStatus
▶	1	Grooming	Full grooming	2500.00	60	Available
	2	Vaccination	Pet vaccines	1500.00	30	Available
	3	Checkup	General health check	1000.00	20	Available
	4	Surgery	Minor surgery	10000.00	120	Unavailable
	5	Training	Pet training	3000.00	90	Available
*	NULL	NULL	NULL	NULL	NULL	NULL

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: `Ctrl+Z`

Result Grid
Form Editor

4. Staff

Limit to 1000 rows

```
1 • SELECT * FROM staff;
```

staffID	fullName	role	phone	email	shift
1	John Silva	Manager	0710000001	john@gmail.com	Morning
2	Peter Perera	Supervisor	0710000002	peter@gmail.com	Evening
3	Anne Dias	Support	0710000003	anne@gmail.com	Morning
4	Kavindu Raj	Admin	0710000004	kavi@gmail.com	Night
5	Ruwan Sen	Trainer	0710000005	ruwan@gmail.com	Morning
NULL	NULL	NULL	NULL	NULL	NULL

5. Operations Manager

Limit to 1000 rows

```
1 • SELECT * FROM OperationsManager;
```

staffID
1
NULL

6. Customer Support Officer

Limit to 1000 rows

```
1 • SELECT * FROM CustomerSupportOfficer;
```

staffID
3
NULL

7. System Administrator

1 • `SELECT * FROM SystemAdministrator;`

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: [I](#)

staffID
4
NULL

Result Grid
Form Editor

8. Booking

1 • `SELECT * FROM Booking;`

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: [I](#)

bookingID	bookingDate	bookingTime	status	customerID	petID	serviceID	staffID
1	2026-05-01	10:00:00	Completed	1	1	1	1
2	2026-05-02	11:00:00	Pending	2	2	2	2
3	2026-05-03	12:00:00	Completed	3	3	3	3
4	2026-05-04	13:00:00	Cancelled	4	4	1	4
5	2026-05-05	14:00:00	Pending	5	5	5	5
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Result Grid
Form Editor

9. Reports

1 • `SELECT * FROM Report;`

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: [I](#)

reportID	reportType	generatedDate	format	adminID	staffID
1	Sales	2026-05-01	PDF	1	1
2	Usage	2026-05-02	Excel	2	2
3	Health	2026-05-03	PDF	3	3
4	Finance	2026-05-04	Word	4	4
5	Summary	2026-05-05	PDF	5	5
NULL	NULL	NULL	NULL	NULL	NULL

Result Grid
Form Editor

10. Admin

1 • `SELECT * FROM Admin;`

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: `Ctrl+Alt+Z`

adminID	fullName	email	password	permissionLevel	status
1	Admin One	admin1@gmail.com	pass	High	Active
2	Admin Two	admin2@gmail.com	pass	Medium	Active
3	Admin Three	admin3@gmail.com	pass	Low	Inactive
4	Admin Four	admin4@gmail.com	pass	High	Active
5	Admin Five	admin5@gmail.com	pass	Medium	Active
NULL	NULL	NULL	NULL	NULL	NULL

Result Grid | Form Editor

Part D – SQL Queries & Outputs

1. SELECT query

1 `SELECT customerID, fullName, email, phone`
 2 `FROM Customer`
 3 `WHERE status = 'Active';`

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: `Ctrl+Alt+Z`

customerID	fullName	email	phone
1	Nimal Perera	nimal@gmail.com	0711111111
2	Kamal Silva	kamal@gmail.com	0722222222
4	Sunil Fernando	sunil@gmail.com	0744444444
5	Amali De Silva	amali@gmail.com	0755555555
NULL	NULL	NULL	NULL

Result Grid | Form Editor

2. JOIN query

The screenshot shows a database query editor with a SQL query and its results. The query is a JOIN query that combines data from the Booking, Customer, Pet, and Service tables. The result grid displays 5 rows of data.

```
1 • SELECT
2     b.bookingID,
3     c.fullName AS CustomerName,
4     p.name AS PetName,
5     s.name AS ServiceName,
6     b.bookingDate,
7     b.status
8 FROM Booking b
9 JOIN Customer c ON b.customerID = c.customerID
10 JOIN Pet p ON b.petID = p.petID
11 JOIN Service s ON b.serviceID = s.serviceID;
```

bookingID	CustomerName	PetName	ServiceName	bookingDate	status
1	Nimal Perera	Max	Grooming	2026-05-01	Completed
2	Kamal Silva	Kitty	Vaccination	2026-05-02	Pending
3	Saman Kumara	Rocky	Checkup	2026-05-03	Completed
4	Sunil Fernando	Milo	Grooming	2026-05-04	Cancelled
5	Amali De Silva	Bella	Training	2026-05-05	Pending

3. Aggregation

The screenshot shows a database query editor with a SQL query that calculates the total revenue from completed bookings. The result grid displays a single row with the total revenue value.

```
1 • SELECT SUM(s.price) AS TotalRevenue
2 FROM Booking b
3 JOIN Service s ON b.serviceID = s.serviceID
4 WHERE b.status = 'Completed';
```

TotalRevenue
3500.00

4. GROUP BY / HAVING

The screenshot shows a SQL IDE with a query editor and a result grid. The query is as follows:

```

1  SELECT
2      s.name AS ServiceName,
3      COUNT(b.bookingID) AS TotalBookings
4  FROM Booking b
5  JOIN Service s ON b.serviceID = s.serviceID
6  GROUP BY s.name
7  HAVING COUNT(b.bookingID) > 1;

```

The result grid shows the following data:

ServiceName	TotalBookings
Grooming	2

5. Subquery

The screenshot shows a SQL IDE with a query editor. The query is as follows:

```

1  SELECT fullName
2  FROM Customer
3  WHERE customerID IN (
4      SELECT customerID
5      FROM Booking
6  )

```

The result grid shows the following data:

ServiceName	TotalBookings
Grooming	2

PART E: Stored Function/Procedure

Get total number of bookings for a given customer:

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0076 seconds.)

```

CREATE FUNCTION GetBookingCount(custID INT)
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE total INT;
    SELECT COUNT(*)
    INTO total
    FROM Booking
    WHERE customerID = custID;
    RETURN total;
END;;

```

```
1
2 DELIMITER //
3 CREATE FUNCTION GetBookingCount(custID INT)
4 RETURNS INT
5 DETERMINISTIC
6 BEGIN
7     DECLARE total INT;
8
9     SELECT COUNT(*) INTO total
10    FROM Booking
11   WHERE customerID = custID;
12
13     RETURN total;
14 END;
15 DELIMITER ;
16
17 • SELECT GetBookingCount(1);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

GetBookingCount(1)
1

Result Grid

PART F: Trigger

This trigger validates the booking status before inserting data. It ensures only predefined values are stored, otherwise it stops the insertion using an error.

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0113 seconds.)

```
CREATE TRIGGER trg_validate_booking_status
BEFORE INSERT ON Booking
FOR EACH ROW

BEGIN IF NEW.status NOT IN ('Pending', 'Confirmed', 'Completed', 'Cancelled')
THEN SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Invalid booking status';
END IF;
END;;
```

Trigger pass:

Limit to 1000 rows

```
1
2 • INSERT INTO Booking VALUES(31,'2026-06-10','10:00','Pending',1,1,1,1);
```

Output

Action Output

#	Time	Action	Message
✓ 1	22:02:03	INSERT INTO Booking VALUES(31,'2026-06-10','10:00','Pending',1,1,1,1)	1 row(s) affected

Trigger fail:

Limit to 1000 rows

```
1
2 • INSERT INTO Booking VALUES(30,'2026-06-10','10:00','WrongStatus',1,1,1,1);
```

Output

Action Output

#	Time	Action	Message
✗ 1	22:04:32	INSERT INTO Booking VALUES(30,'2026-06-10','10:00','WrongStatus',1,1,1,1)	Error Code: 1644. Invalid booking status